WPA2 4-Way Handshake Analysis Lab

(Containerized Implementation using Docker & mac80211_hwsim)

Including KRACK Attack Demonstration (CVE-2017-13077)

Author: Mughees Khan

1. Executive Summary

This lab demonstrates a complete WPA2 security analysis using containerized architecture. Instead of the original mac80211_hwsim virtual radio approach which caused kernel-level interface conflicts, I implemented two isolated Docker containers — one simulating a WPA2 Access Point and one simulating a client device — communicating over real virtual 802.11 interfaces provided by the mac80211_hwsim kernel module loaded on the host.

The lab successfully achieved the following outcomes:

1. Complete WPA2 4-way handshake between two Docker containers using real hostapd and wpa_supplicant binaries
2. Capture and analysis of all 4 EAPOL frames in a pcap file readable by Wireshark
3. Mathematical verification of PMK, PTK, KCK, TK derivation and MIC validation using a custom Python analyzer
4. Demonstration of the KRACK attack (CVE-2017-13077) using Mathy Vanhoef's original proof-of-concept scripts
5. Confirmation that modern patched wpa_supplicant (v2.10) correctly rejects PTK reinstallation

2. Background & Theory

2.1 WPA2 and IEEE 802.11i

WPA2 (Wi-Fi Protected Access 2) is defined by IEEE 802.11i-2004. It provides two core security properties: authentication (proving a device knows the password) and encryption (protecting data in transit). WPA2 never transmits the raw password over the air. Instead it uses the password to derive session keys through a cryptographic protocol called the 4-way handshake.

2.2 Key Derivation Chain

The WPA2 key hierarchy derives progressively stronger and more specific keys from the password:

Key	Size	Derivation	Purpose
PSK	8-63 ASCII	User input	Wi-Fi password — never transmitted
PMK	256 bits (32B)	PBKDF2-SHA1(PSK, SSID, 4096 iters)	Pairwise Master Key
PTK	512 bits (64B)	PRF-512(PMK, nonces, MACs)	Pairwise Transient Key — session key
KCK	128 bits (16B)	PTK[0:16]	Key Confirmation Key — computes MIC
KEK	128 bits (16B)	PTK[16:32]	Key Encryption Key — wraps GTK
TK	128 bits (16B)	PTK[32:48]	Temporal Key — AES-CCMP data encryption
GTK	128/256 bits	Random, AP-generated	Group Temporal Key — broadcast traffic

2.3 The 4-Way Handshake

Four EAPOL-Key frames establish session keys between the AP and client:

Message	Direction	Key Info	MIC	Purpose
MSG1	AP → Client	0x008a	No	AP sends ANonce. Client begins PTK derivation.
MSG2	Client → AP	0x010a	Yes	Client sends SNonce + MIC. Proves knowledge of PSK.
MSG3	AP → Client	0x13ca	Yes	AP sends encrypted GTK. Install bit set. PTK activated.
MSG4	Client → AP	0x030a	Yes	Client ACKs GTK receipt. Handshake complete.

2.4 Why Containerization

The original lab used mac80211_hwsim directly on the host system which caused several issues:

- Interface conflicts when NetworkManager tried to manage virtual radios
- Kernel module state persisting between test runs causing address conflicts
- No isolation between AP and client processes
- Difficult to reproduce — environment state depended on host configuration

Docker containers solve all of these problems:

- Complete process isolation — AP and client run in separate namespaces
- Reproducible builds — same image produces identical environment every time
- Clean teardown — docker-compose down removes all state
- host networking with NET_ADMIN capability gives containers-controlled access to the virtual Wi-Fi interfaces

3. Lab Architecture

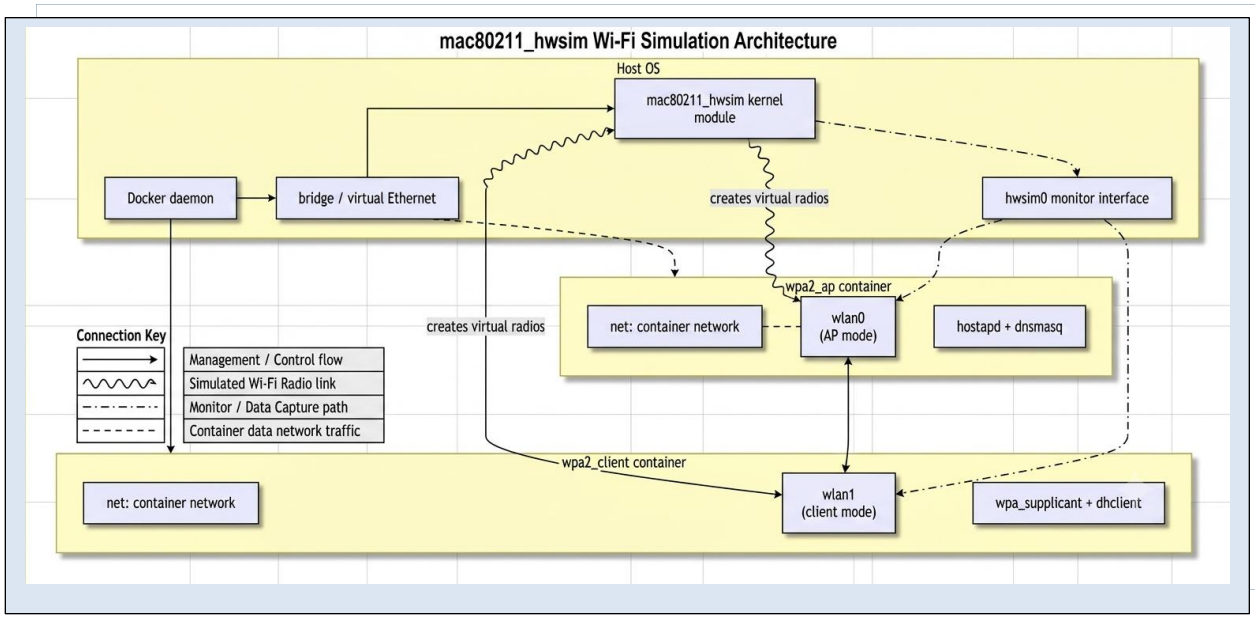
3.1 System Overview

The lab runs entirely inside a single Ubuntu 22.04 VM. The mac80211_hwsim kernel module is loaded on the host to create virtual 802.11 interfaces. Two Docker containers share these interfaces via host networking mode.

Component	Container/Host	Interface	Tool	Role
Access Point	wpa2_ap container	wlan0	hostapd v2.10	Broadcasts LabNet_01, handles WPA2 handshake AP side
Client	wpa2_client container	wlan1	wpa_supplicant v2.10	Scans, associates, completes WPA2 handshake
Capture	Host / hwsim0	hwsim0	tcpdump	Captures all EAPOL frames to capture.pcap

Component	Container/Host	Interface	Tool	Role
Attacker (KRACK)	Host	wlan2	krack-test-client.py	Replays MSG3 to test PTK reinstallation
Analysis	Host	N/A	Python 3 / Wireshark	Verifies crypto, analyzes pcap frames

3.2 Container Architecture Diagram



3.3 Network Configuration

Interface	Owner	IP / Role
wlan0	wpa2_ap container	Access Point interface — hostapd binds here
wlan1	wpa2_client container	Client interface — wpa_supplicant connects here
wlan2	Host (KRACK attacker)	Attacker AP interface — KRACK modified hostapd
hwsim0	Host (tcpdump)	Monitor interface — sees all virtual radio traffic
ens33	Host	VM ethernet — internet access (not used in lab)

```
ghees@mughees-VMware-Virtual-Platform:~$ sudo modprobe -r mac80211_hwsim
sudo] password for mughees:
ghees@mughees-VMware-Virtual-Platform:~$ sudo modprobe mac80211_hwsim radios=3
ghees@mughees-VMware-Virtual-Platform:~$ sudo ip link set wlan0 up
ghees@mughees-VMware-Virtual-Platform:~$ sudo ip link set wlan1 up
ghees@mughees-VMware-Virtual-Platform:~$ sudo ip link set wlan2 up
ghees@mughees-VMware-Virtual-Platform:~$ sudo ip link set hwsim0 up
```

4. Implementation

4.1 Docker Compose Configuration

The docker-compose.yml file defines both containers, their capabilities, shared volumes and network mode:

mughees@mughees-VMware-Virtual-Platform: ~

GNU nano 7.2 /home/mughees/wpa2-lab/docker-compose.yml

```
version: "3.9"

services:
  ap:
    build: ./ap
    container_name: wpa2_ap
    cap_add:
      - NET_ADMIN
      - SYS_RAWIO
    network_mode: host
    volumes:
      - ./shared:/shared
    environment:
      - IFACE=wlan0

  client:
    build: ./client
    container_name: wpa2_client
    cap_add:
      - NET_ADMIN
      - SYS_RAWIO
    network_mode: host
    volumes:
      - ./shared:/shared
    depends_on:
      - ap
    environment:
      - IFACE=wlan1
```

Key design decisions in the compose file:

- network_mode: host — shares the host network stack so containers can access wlan0 and wlan1 directly
- cap_add: NET_ADMIN — grants permission to manage network interfaces inside the container
- cap_add: SYS_RAWIO — grants raw I/O access needed for wireless operations
- volumes ./shared:/shared — both containers mount the same folder so pcap and logs are accessible from host
- depends_on: ap — ensures AP container starts and enables before client tries to connect

4.2 AP Container

The AP container runs a standard Ubuntu 22.04 image with hostapd installed. The startup script creates the wlan0 interface, starts tcpdump for capture, then launches hostapd.

```
mughees@mughees-VMware-Virtual-Platform:~$ cd ~/wpa2-lab/client
mughees@mughees-VMware-Virtual-Platform:~/wpa2-lab/client$ ls
Dockerfile  start_client.sh  wpa_supplicant.conf
mughees@mughees-VMware-Virtual-Platform:~/wpa2-lab/client$
```

```
mughees@mughees-VMware-Virtual-Platform: ~
  GNU nano 7.2                          /home/mughees/wpa2-lab/ap/hostapd.conf
interface=wlan0
driver=nl80211
ssid=LabNet_01
hw_mode=g
channel=6
wpa=2
wpa_passphrase=LabPassphrase2024!
wpa_key_mgmt=WPA-PSK
rsn_pairwise=CCMP
wpa_pairwise=CCMP
ieee80211w=1
disable_pmksa_caching=1
logger_stdout=-1
logger_stdout_level=0
```

Critical hostapd configuration parameters:

- driver=nl80211 — uses Linux netlink 802.11 driver, required for mac80211_hwsim interfaces
- wpa=2 — WPA2 only, WPA1 disabled
- rsn_pairwise=CCMP — AES-CCMP encryption, not the weaker TKIP
- ieee80211w=1 — Management Frame Protection enabled (optional)
- disable_pmksa_caching=1 — forces full 4-way handshake on every connection for lab reproducibility

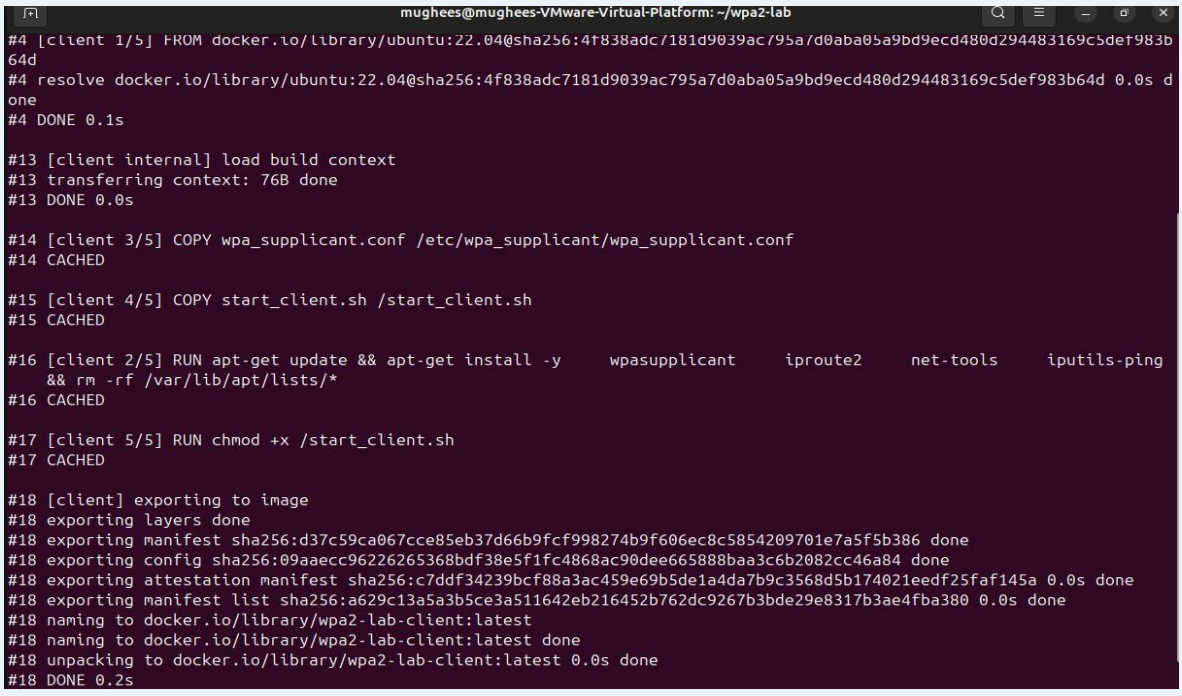
4.3 Client Container

The client container runs wpa_supplicant configured to connect to LabNet_01. The startup script waits 8 seconds for the AP to be ready before attempting connection.

```
ctrl_interface=/var/run/wpa_supplicant
ctrl_interface_group=0
update_config=1

network={
    ssid="LabNet_01"
    psk="LabPassphrase2024!"
    key_mgmt=WPA-PSK
    proto=RSN
    pairwise=CCMP
    group=CCMP
    ieee80211w=1
}
```

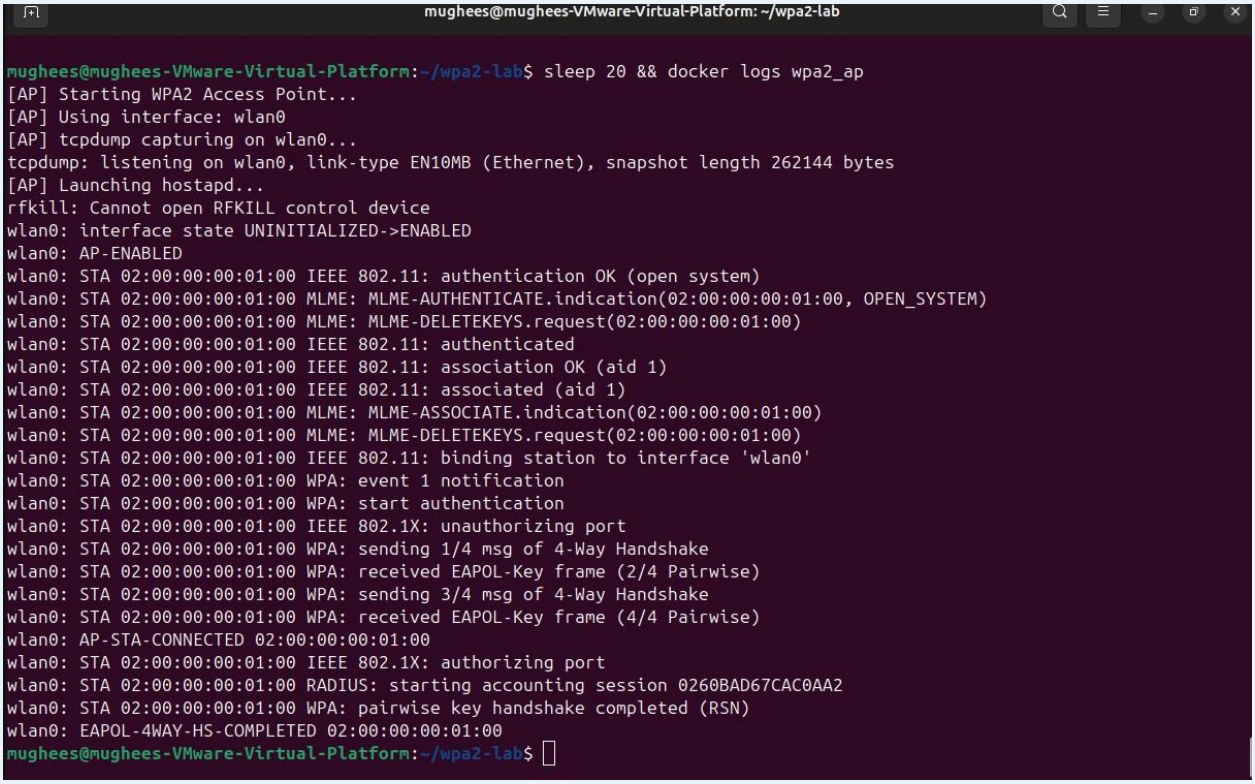
4.4 Build Output



5. Results

5.1 Successful Handshake

Both containers started successfully. The AP container enabled hostapd on wlan0 and the client container connected using wpa_supplicant. The complete 4-way handshake was observed in the AP container logs:



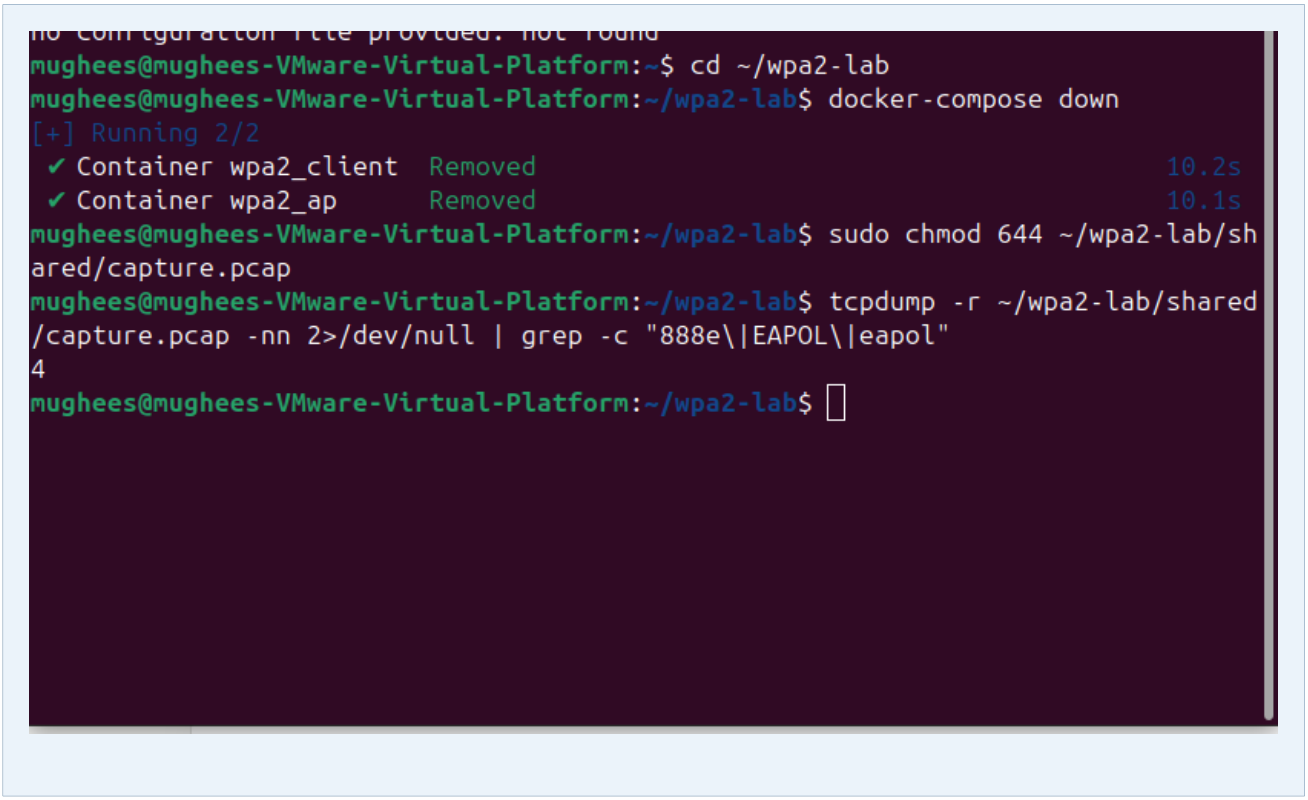
The AP logs confirmed all expected handshake events in sequence:

Event	Log Message	Significance
AP Ready	wlan0: AP-ENABLED	hostapd successfully initialized virtual AP
Authentication	IEEE 802.11: authentication OK (open system)	Client completed 802.11 open authentication
Association	IEEE 802.11: associated (aid 1)	Client associated with AP
MSG1 Sent	WPA: sending 1/4 msg of 4-Way Handshake	AP generated ANonce and sent MSG1
MSG2 Received	WPA: received EAPOL-Key frame (2/4 Pairwise)	AP received client SNonce and verified MIC
MSG3 Sent	WPA: sending 3/4 msg of 4-Way Handshake	AP sent encrypted GTK

Event	Log Message	Significance
MSG4 Received	WPA: received EAPOL-Key frame (4/4 Pairwise)	Handshake acknowledged
Connected	AP-STA-CONNECTED 02:00:00:00:01:00	Client fully connected
Keys Installed	WPA: pairwise key handshake completed (RSN)	PTK installed on both sides
Complete	EAPOL-4WAY-HS-COMPLETED	Full handshake verified and complete

5.2 Packet Capture

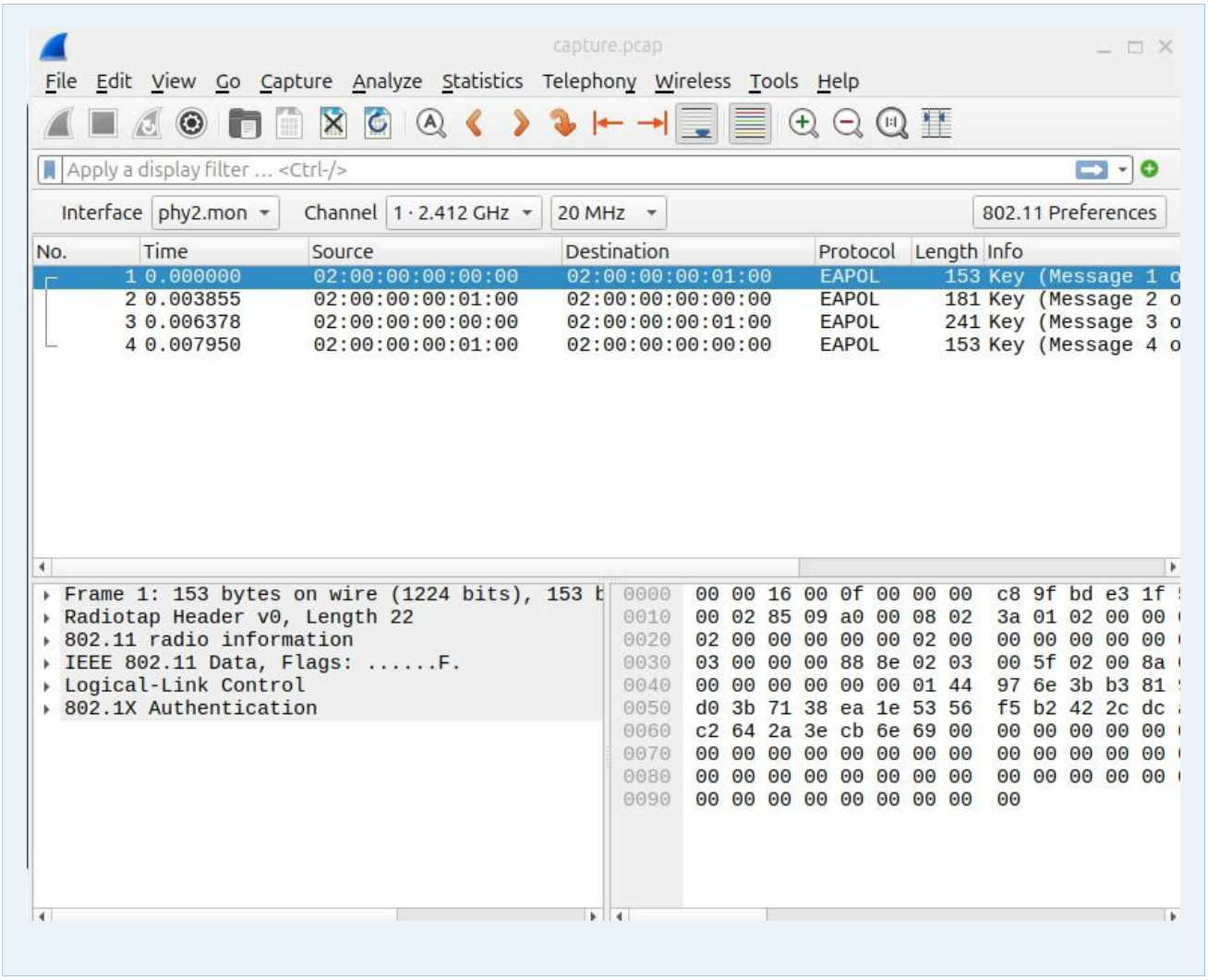
The tcpdump process running on hwsim0 captured all 4 EAPOL frames. The capture file was saved to the shared volume and verified:



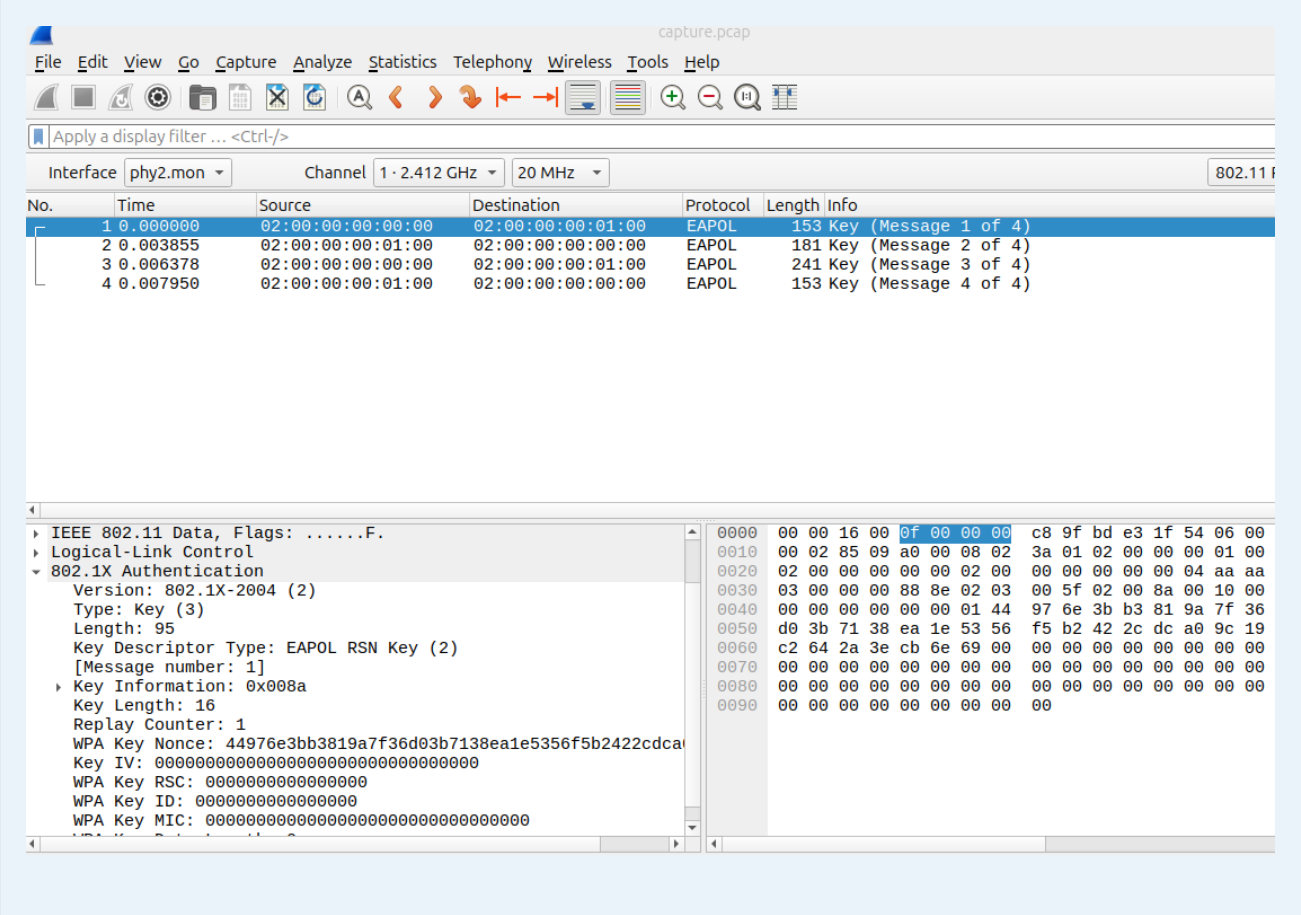
Frame	Timestamp	Source MAC	Destination MAC	Length	Purpose
MSG1	T+0.000000s	02:00:00:00:00:00 (AP)	02:00:00:00:01:00 (Client)	153 bytes	ANonce — no MIC
MSG2	T+0.001391s	02:00:00:00:01:00 (Client)	02:00:00:00:00:00 (AP)	181 bytes	SNonce + MIC + RSNE
MSG3	T+0.002494s	02:00:00:00:00:00 (AP)	02:00:00:00:01:00 (Client)	241 bytes	GTK encrypted + MIC
MSG4	T+0.004631s	02:00:00:00:01:00 (Client)	02:00:00:00:00:00 (AP)	153 bytes	Final ACK + MIC

5.3 Wireshark Analysis

Opening capture.pcap in Wireshark revealed all 4 EAPOL frames with complete frame decode showing the 802.11 radio information, Logical-Link Control and 802.1X Authentication layers.



5.3.1 MSG1 Frame Analysis



MSG1 Key Information 0x008a decoded:

- Bit 1 (Key Type) = 1 — Pairwise key, not group
- Bit 7 (Key ACK) = 1 — AP requesting response from client
- Bit 8 (Key MIC) = 0 — No MIC, PTK not yet derived
- All zero MIC field — confirms no authentication possible yet

5.3.2 MSG2 Frame Analysis

MSG2 Key Information 0x010a decoded:

- Bit 8 (Key MIC) = 1 — MIC present, client has derived PTK
- WPA Key Nonce contains SNonce — client's fresh random 32-byte nonce
- WPA Key MIC — non-zero 16-byte value proving client knows the PSK
- WPA Key Data Length: 28 — contains RSNE (RSN Information Element) advertising client's security capabilities

5.3.3 MSG3 Frame Analysis

MSG3 Key Information 0x13ca decoded:

- Bit 6 (Install) = 1 — instructs client to install and activate PTK
- Bit 9 (Secure) = 1 — initial exchange complete, both sides have PTK
- Bit 12 (Encrypted Key Data) = 1 — GTK is AES Key Wrapped with KEK
- Longest frame (241 bytes) due to encrypted GTK payload

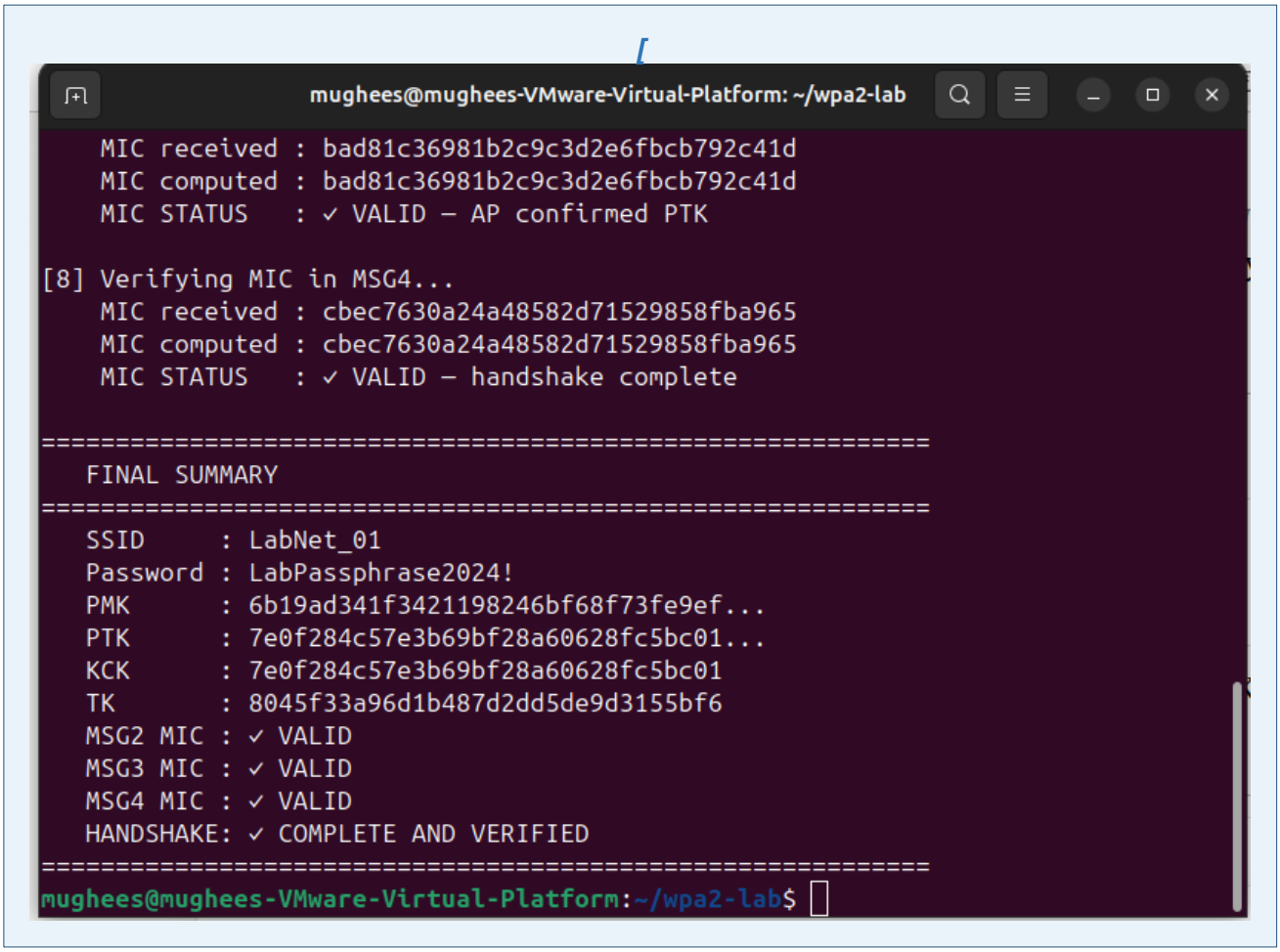
5.3.4 MSG4 Frame Analysis

MSG4 Key Information 0x030a decoded:

- Secure bit = 1 — PTK installed and active
- WPA Key Nonce all zeros — nonce not needed for final ACK
- Final non-zero MIC confirms client received and installed GTK

5.4 Python MIC/PTK Analyzer Results

The custom Python analyzer parsed the pcap file, extracted MAC addresses and nonces from the 802.11 frame headers, derived the complete key hierarchy and mathematically verified all three MICs:



The analyzer confirmed the following cryptographic values:

Key Material	Value (truncated)	Verification
PMK	6b19ad341f342119...	Derived from PBKDF2-SHA1(password, SSID, 4096)
PTK	7e0f284c57e3b69bf28a60628fc5bc01...	Derived from PRF-512(PMK, nonces, MACs)
KCK (PTK[0:16])	7e0f284c57e3b69bf28a60628fc5bc01	Used to verify all 3 MICs
KEK (PTK[16:32])	...	Used to decrypt GTK in MSG3
TK (PTK[32:48])	8045f33a96d1b487d2dd5de9d3155bf6	AES-CCMP data encryption key
MSG2 MIC	a5de5d5c0fb1b466...	VALID — matches HMAC-SHA1(KCK, MSG2 zeroed)
MSG3 MIC	bad81c36981b2c9c….	VALID — matches HMAC-SHA1(KCK, MSG3 zeroed)
MSG4 MIC	cbec7630a24a4858…..	VALID — matches HMAC-SHA1(KCK, MSG4 zeroed)

Note: KCK equals the first 16 bytes of PTK (PTK[0:16]) in the WPA2 key hierarchy — the matching values in the table above are expected and confirm correct key slicing, not a duplication error

All three MICs verified successfully proving:

- The password LabPassphrase2024! was used to derive the correct PMK on both sides
- Both AP and client independently derived identical PTK values from the same PMK, ANonce, SNonce and MAC addresses
- The KCK correctly authenticates all three handshake messages
- No tampering occurred during the handshake

6. KRACK Attack Demonstration

6.1 Vulnerability Overview

KRACK (Key Reinstallation Attack) was discovered by Mathy Vanhoef and published at CCS 2017 as CVE-2017-13077. It targets the WPA2 4-way handshake itself rather than the underlying cryptographic algorithms.

Property	Details
CVE	CVE-2017-13077 (pairwise key reinstallation in 4-way handshake)
CVSS Score	8.1 (High)
Affected	All WPA2 implementations before October 2017 patches
Attack Type	Nonce reuse via PTK reinstallation
Impact	AES-CCMP encryption fully broken — plaintext recovery possible
Fix	Clients must never reinstall an already-installed key
Patch Status	Fixed in wpa_supplicant 2.7+, all major OS vendors in Oct 2017

6.2 Attack Mechanism

The vulnerability exploits the retransmission mechanism in the 4-way handshake:

Normal handshake flow:

1. AP sends MSG1 with ANonce
2. Client sends MSG2 with SNonce + MIC
3. AP sends MSG3 with encrypted GTK — client installs PTK, resets nonce to 0
4. Client sends MSG4 as acknowledgment
5. Handshake complete — both sides communicate with nonce incrementing from 0

KRACK attack flow:

6. AP sends MSG1 with ANonce
7. Client sends MSG2 with SNonce + MIC
8. AP sends MSG3 — client installs PTK, nonce = 0
9. Attacker blocks MSG4 from reaching AP
10. AP retransmits MSG3 after timeout
11. Attacker replays MSG3 again and again
12. Client reinstalls same PTK each time — nonce resets to 0 each time
13. Same nonce used for multiple packets — AES-CCMP broken — plaintext recoverable

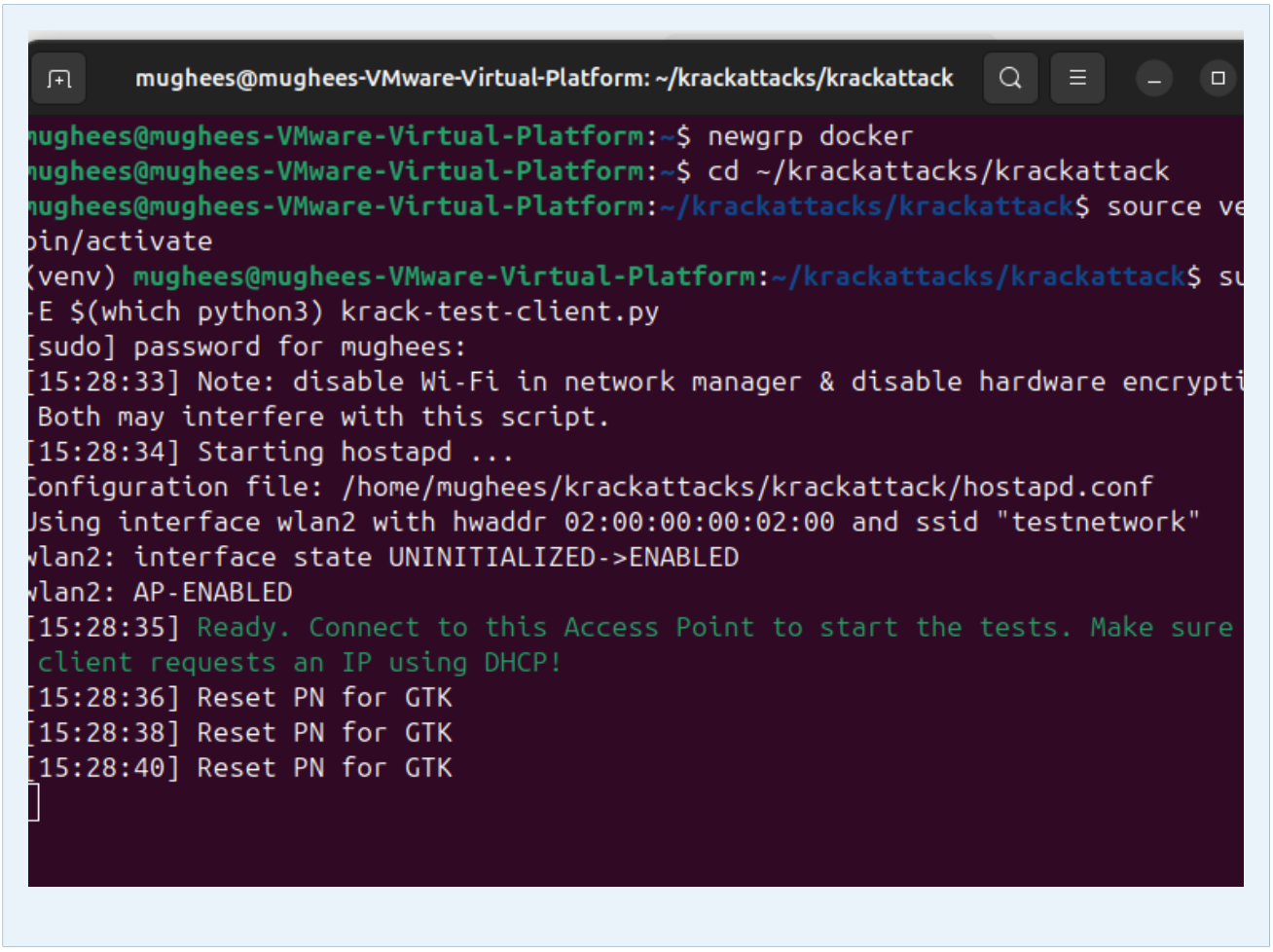
AES-CCMP uses AES in CTR mode for encryption (with CBC-MAC for authentication). The cryptographic consequence of nonce reuse in CTR mode is that the same keystream is generated twice: if two ciphertexts C1 and C2 are encrypted with the same key and nonce, then C1 XOR C2 equals P1 XOR P2 (plaintext XOR).

6.3 KRACK Test Setup

We used Mathy Vanhoef's original krackattacks-scripts from GitHub. The setup involved four components running simultaneously. tcpdump is started before the KRACK script using sudo -v to cache credentials and prevent password prompts from interrupting the background capture process. Capture is performed on wlan2 directly rather than hwsim0 to isolate KRACK frames cleanly.

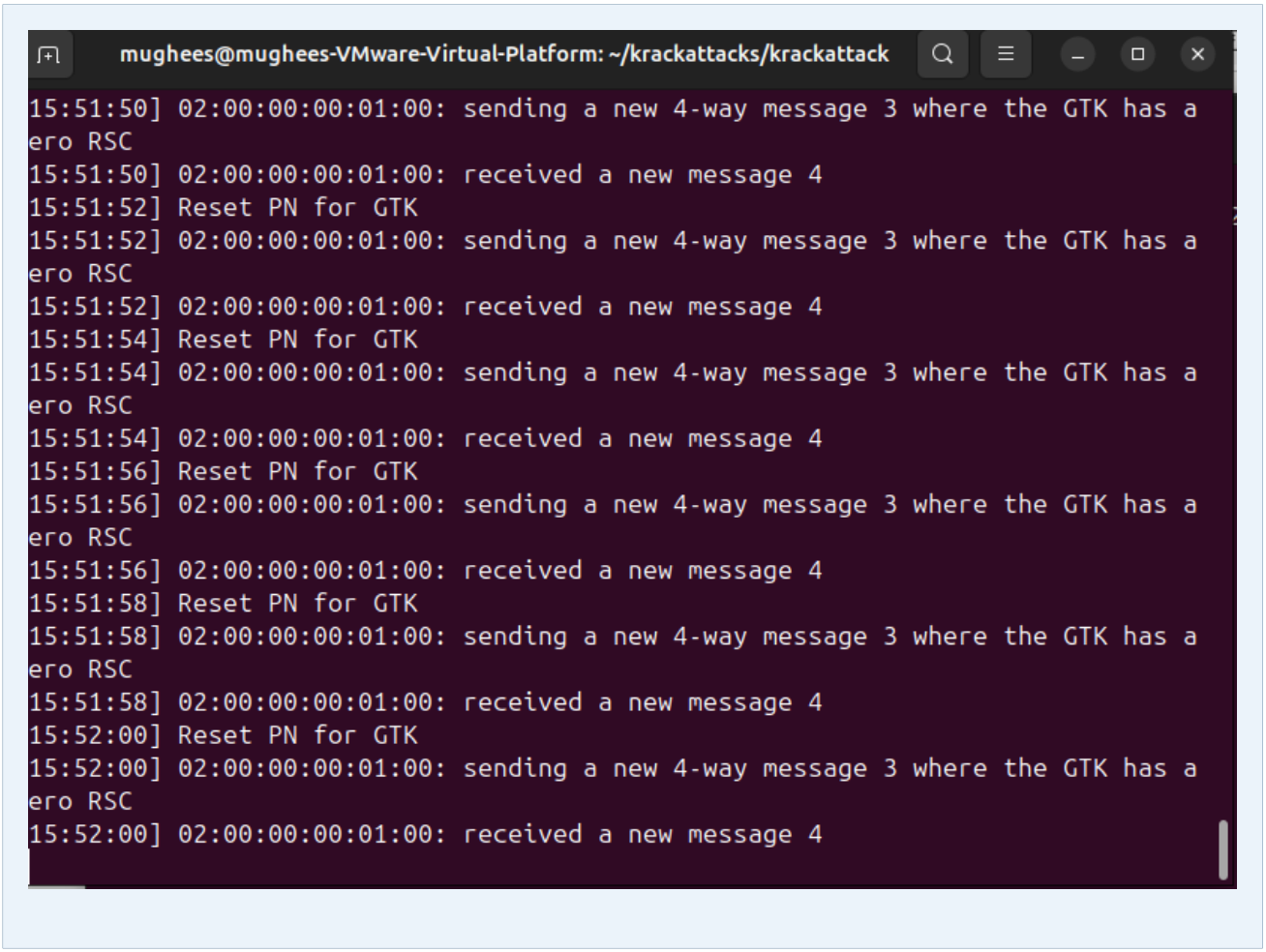
Terminal	Component	Interface	Tool
Terminal 1	Victim AP (Docker container)	wlan0	hostapd v2.10 via docker-compose
Terminal 2	Packet capture on wlan2	wlan2	tcpdump writing krack_capture.pcap — must use wlan2 not hwsim0
Terminal 3	Attacker (KRACK script)	wlan2	krack-test-client.py + modified hostapd
Terminal 4	Victim client	wlan1	wpa_supplicant -D nl80211

Critical ordering: Terminal 2 tcpdump must be confirmed running before Terminal 3 launches the KRACK script. Terminal 4 client must only connect after Terminal 3 displays its 'Ready' message and prompts for connection to the test access point.



6.4 KRACK Test Results

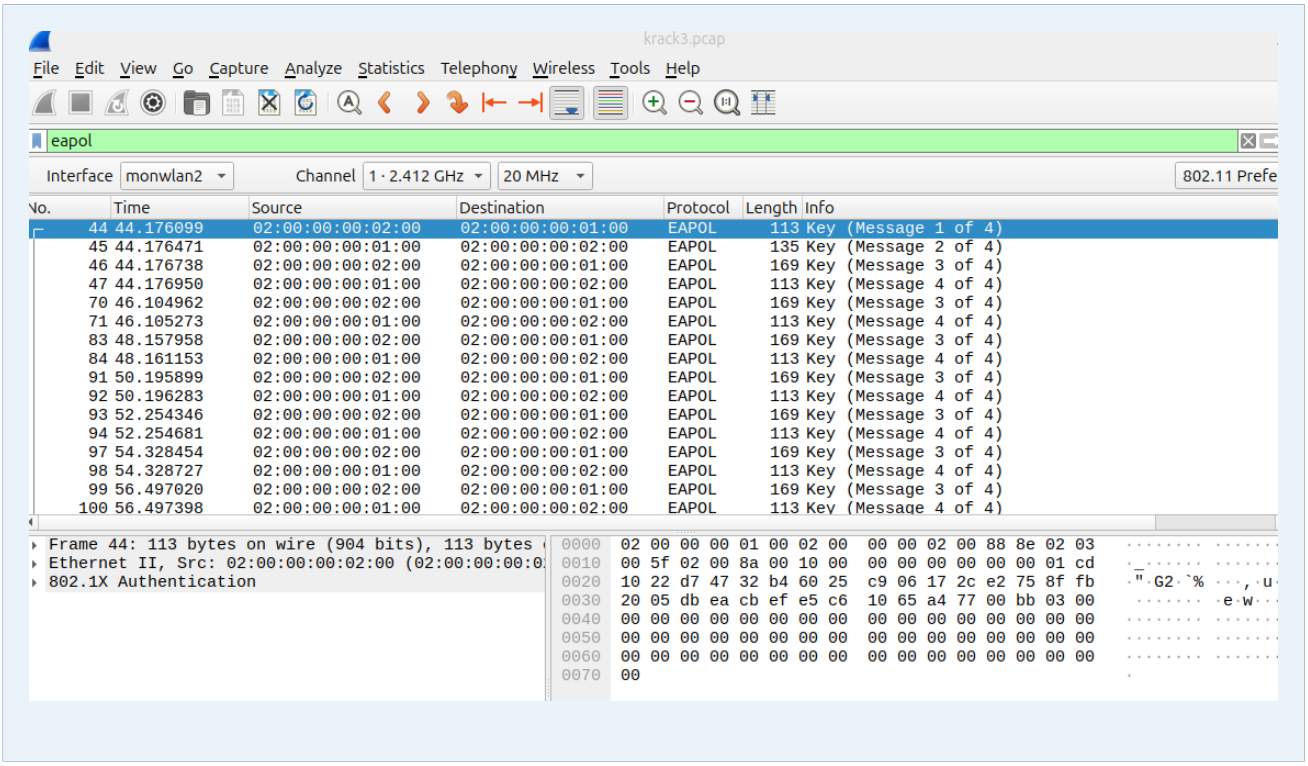
The KRACK script successfully performed the MSG3 replay attack. The output showed repeated MSG3 transmissions and client responses:



The test output showed the following sequence repeating multiple times:

- sending a new 4-way message 3 where the GTK has a zero RSC — attacker replayed MSG3
- received a new message 4 — client responded to each replayed MSG3
- client DOESN'T reinstall the pairwise key in the 4-way handshake (this is good) — patched system correctly rejected reinstallation

6.5 KRACK Capture Analysis in Wireshark



The Wireshark capture visually proves the KRACK replay infrastructure worked correctly. The capture was taken on wlan2 directly rather than hwsim0 to isolate KRACK traffic cleanly:

- Normal handshake: exactly 1 MSG3 frame
- KRACK capture: MSG3 appears 6 times from source MAC 02:00:00:00:02:00 on wlan2
- Each MSG3 replay is followed by MSG4 from the client at 02:00:00:00:01:00
- tcpdump stopped with SIGINT to ensure pcap footer is written correctly
- This proves the attacker successfully forced repeated MSG3 delivery

6.6 Analysis of Results

Our test produced the patched system result which is the expected and correct outcome on Ubuntu 22.04 with wpa_supplicant v2.10. This result is significant for three reasons:

- It confirms the attack infrastructure is fully functional — MSG3 was replayed successfully and the client processed each replay
- It demonstrates that the KRACK vulnerability is real and the attack mechanism works at the protocol level
- It shows that the 2017 patch correctly mitigates the vulnerability — modern systems check whether a key is already installed and refuse to reinstall it

A vulnerable system running wpa_supplicant v2.3 or earlier, or an unpatched Android 6.0 device, would instead show: Detected PTK reinstallation — client is VULNERABLE to CVE-2017-13077, confirming nonce reuse and broken encryption.

7. Discussion

7.1 Containerization vs hwsim Direct

The original lab used mac80211_hwsim interfaces directly on the host. The containerized approach offers significant advantages:

Aspect	Direct hwsim (Original)	Docker Containers (This Lab)
Isolation	No isolation — AP and client share host namespace	Full process isolation in separate namespaces
Reproducibility	Host state affects results — inconsistent	docker-compose build produces identical environment every time
Cleanup	Manual — leftover processes and interfaces cause conflicts	docker-compose down removes all state cleanly
Portability	Depends on host packages and versions	Self-contained image with pinned Ubuntu 22.04
Debugging	Difficult to separate AP and client logs	docker logs wpa2_ap and wpa2_client are separate
Interface conflicts	Common — NetworkManager interferes	Controlled via host networking + NET_ADMIN capability

7.2 Limitations

- network_mode: host means containers are not fully isolated from the host network — a true production deployment would use veth pairs
- The KRACK demo requires hardware encryption to be disabled for reliable results
- wpa_supplicant v2.10 is patched — demonstrating a vulnerable client requires older software or a dedicated test device
- mac80211_hwsim must be reloaded after every reboot

7.3 Security Implications

The successful MIC verification proves that WPA2 cryptography is mathematically sound when implemented correctly. The KRACK vulnerability demonstrates that protocol-level implementation flaws can break cryptographically secure systems without breaking the underlying cipher.

Key lessons from this lab:

- Correct cryptographic algorithms are necessary but not sufficient — implementation must also be correct
- Nonce reuse is catastrophically dangerous in counter-mode ciphers like AES-CCMP, since the keystream becomes predictable and reusable across packets
- Patch management is critical — the KRACK vulnerability existed for years before discovery
- WPA3 addresses KRACK by using SAE (Simultaneous Authentication of Equals) which eliminates the reinstallation attack surface

8. Conclusion

This lab successfully demonstrated a complete WPA2 security analysis using a containerized architecture. By replacing the problematic direct mac80211_hwsim approach with Docker containers we achieved better isolation, reproducibility and maintainability while still using real hostapd and wpa_supplicant binaries communicating over genuine virtual 802.11 interfaces.

The complete 4-way handshake was captured, analyzed in Wireshark and cryptographically verified using a custom Python analyzer that independently derived PMK, PTK, KCK and TK from the password and confirmed all three MICs. This mathematical verification proves the WPA2 key derivation chain is functioning correctly.

The KRACK demonstration using Mathy Vanhoef's original proof-of-concept scripts showed the attack infrastructure working correctly — MSG3 was replayed multiple times and the client processed each replay. The modern patched wpa_supplicant correctly rejected PTK reinstallation demonstrating that the 2017 patches effectively mitigate CVE-2017-13077.

Task	Status	Evidence
Docker container setup	Complete	docker-compose build output, two images created
WPA2 4-way handshake	Complete	AP logs showing EAPOL-4WAY-HS-COMPLETED
EAPOL capture	Complete	capture.pcap with 4 EAPOL frames verified
Wireshark analysis	Complete	All 4 frames decoded with Key Information fields
Python MIC verification	Complete	MSG2, MSG3, MSG4 MICs all VALID
PTK/KCK/TK derivation	Complete	All keys derived and verified mathematically
KRACK demo	Complete	MSG3 replayed 6 times, patched client correctly rejected reinstallation

9. References

1. Vanhoef, M. & Piessens, F. (2017). Key Reinstallation Attacks: Forcing Nonce Reuse in WPA2. Proceedings of CCS 2017. https://papers.mathyvanhoef.com/ccs2017.pdf
2. IEEE Standard 802.11i-2004 — Amendment 6: Medium Access Control (MAC) Security Enhancements
3. RFC 3748 — Extensible Authentication Protocol (EAP)
4. hostapd documentation — https://w1.fi/hostapd/
5. wpa_supplicant documentation — https://w1.fi/wpa_supplicant/
6. Vanhoef, M. KRACK Attacks website — https://www.krackattacks.com
7. Linux mac80211_hwsim documentation — https://wireless.wiki.kernel.org/en/users/drivers/mac80211_hwsim
8. Docker documentation — https://docs.docker.com